SETU AI (CORTEX AI)

Complete System Design & Multi-Agent Architecture Document

Core Engineering Philosophy & Design Specification

Setu AI is an enterprise-grade multi-agent autonomous AI platform featuring distributed microservices, a centralized API Gateway, LangGraph dynamic state orchestration, multimodal retrieval-augmented generation (RAG), live code artifact sandbox generation, and high-performance in-memory session caching.

MICROSERVICES & GATEWAY

Distributed Express services coordinated via a centralized HTTP Gateway with token authentication and header decoration.

LANGGRAPH STATE GRAPH

Autonomous cyclical router dispatching specialized agents for Chat, Web Search, Code Artifacts, PDF/PPT, and Vision.

RAG & VECTOR SEARCH

Binary parsing with pdf-parse v2, LangChain recursive chunking, Gemini 3072-dim embeddings, and Qdrant vector database.

DISTRIBUTED STORAGE & CACHE

Redis cluster for sliding-window rate limiting, token session store, and AWS S3 with secure Presigned URL generation.

1. Executive Vision & Architectural Philosophy

Modern agentic applications require high throughput, multi-tenant state isolation, and deterministic execution paths. Traditional monolithic architectures collapse under the weight of streaming token consumption, heavy document processing, vector embeddings, and disparate external API latencies.

ARCHITECTURAL PRINCIPLE
Design Rule: Separate fast I/O and user-facing transactions from long-running, resource-intensive AI graph traversals and asset generation tasks.

Core Architectural Principles

• **Resilience & Microservices Isolation:** A failure in document parsing or a slow LLM call in the Agent service must never degrade chat retrieval, user authentication, or payment settlement.

• **Modality-Driven Dynamic Routing:** The system inspects user payload headers, file MIME types, and prompt semantics to route requests to specialized agents without rigid monolithic conditionals.

• **Deterministic Token & Cost Economics:** Rate limiting and credit metering occur at the edge and per-agent level to prevent abusive queries and budget exhaustion.

• **Stateless Workers with Stateful Cache:** Worker services remain horizontally scalable and stateless, while Redis maintains session contexts, conversation caching, and sliding-window rate counters.

2. Distributed Microservices Breakdown

The platform is composed of six distinct distributed services collaborating over a low-latency network:

• **API Gateway (Port 8000):** Single public entrypoint for client traffic. Manages CORS, cookie-based session verification, rate limits, and proxies requests to downstream microservices using express-http-proxy without corrupting multipart streams.

• **Auth Service (Port 8001):** Verifies Firebase OAuth tokens, provisions user records in MongoDB Atlas, generates cryptographic session tokens in Redis (7-day TTL), and manages user credit balances.

• **Chat Service (Port 8002):** Maintains conversational state, conversation thread hierarchies, and message histories across User and Assistant turns with timestamps and artifact linkages.

• **Agent Service (Port 8003):** The multi-agent execution core. Hosts the LangGraph StateGraph, vector indexing pipelines, external LLM clients, PDFKit document generation, and AWS S3 integration.

• **Billing Service (Port 8004):** Handles Razorpay payments, webhook verification, tier subscriptions (Free/Pro), and transactional credit top-ups.

• **Frontend Client (Port 5173):** Single Page Application built with React, Redux Toolkit, Tailwind CSS, and Monaco Editor for real-time code artifact previewing.

3. LangGraph Multi-Agent Orchestration Layer

At the center of the Agent Service is a cyclical StateGraph compiled via @langchain/langgraph. Rather than relying on simple prompt chaining, execution flows through an explicit state machine:

ARCHITECTURAL PRINCIPLE
Execution Flow: [Start] -> Router Node -> Conditional Dispatch -> [Specialized Agent Node] -> [End]

Dynamic Router Logic

• **Modality Priority:** If a binary payload is uploaded, router immediately dispatches to pdfRag (application/pdf) or imageAnalyzer (image/*). User files take absolute precedence over default text generation.

• **Agent Selector Override:** When the user explicitly selects an agent from the UI (Chat, Coding, PDF, PPT, Vision, Search), the router honors their explicit choice.

• **LLM Semantic Classifier:** In Auto mode, a specialized router model classifies intent into the single most optimal agent based on query semantics.

Specialized Agent Specifications

• **pdfAgent:** Synthesizes professional multi-section documents using PDFKit vector drawing primitives, custom typography, headings, bulleted points, and streams directly to AWS S3.

• **pptAgent:** Leverages PptxGenJS to create 16:9 widescreen presentations with color-coded title slides, structured content cards, Aptos typography, and automatic pagination.

• **codingAgent:** Generates executable multi-file code artifacts (HTML, CSS, JS) that render live inside the frontend Monaco editor and sandboxed iframe previewer.

• **searchAgent:** Queries Tavily real-time internet search, parses external sources, and injects clean context into chatAgent for up-to-the-minute factual answers.

• **visionAgent:** Synthesizes high-detail image prompts using an elite prompt-engineering LLM, invokes Pollinations AI generation, and stores artifacts on S3.

4. RAG Pipeline & Vector Search Engineering

Document comprehension is powered by a grounded Retrieval-Augmented Generation (RAG) pipeline designed to prevent hallucinations:

Step-by-Step Document Ingestion Pipeline

• **1. Binary Stream Ingestion:** Uploaded PDF files are streamed through the gateway with parseReqBody disabled to preserve boundary headers, and buffered via Multer disk storage.

• **2. Text Layer Decoding:** pdf-parse v2 decodes binary streams into a normalized string representation, extracting layout and textual glyphs into memory.

• **3. Recursive Chunking:** LangChain RecursiveCharacterTextSplitter fragments text into 1000-character chunks with a 200-character overlapping sliding window to preserve cross-boundary context.

• **4. Dense Vector Embedding:** Google Generative AI Embeddings transforms each chunk into a high-dimensional dense vector representation.

• **5. Vector Indexing in Qdrant:** Chunks are indexed into a dynamic collection in Qdrant Vector Cloud for cosine similarity matching.

• **6. Grounded Context Synthesis:** The user question triggers similarity search (top k=5). Retrieved contexts are bound into a strict SystemMessage instructing the model to answer exclusively from the document.

• **7. Guaranteed Cleanup:** Temporary files are deleted in a guarded finally block, preventing disk exhaustion on host servers.

5. Cloud Storage & Artifact Presigning

Generated assets (PDFs, PPTXs, images) are never stored on local server disks. Instead, they are streamed as binary buffers directly to AWS S3 in region ap-south-1. Download URLs are delivered to users via time-limited, cryptographically signed AWS S3 Presigned URLs (GetObjectCommand) with configurable expiration.

6. Production Verification & Status

All distributed components have been validated with active health checks: Gateway (8000), Auth (8001), Chat (8002), Agent (8003), Billing (8004), Frontend (5173), MongoDB Atlas, Qdrant Cloud, and AWS S3.